

UCS645

PARALLEL & DISTRIBUTED COMPUTING



ASSIGNMENT 7:

Submitted by: Sujal Mallick (102303266)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Sum of First N Integers

1. Objective

To implement CUDA kernels where different threads perform different tasks to compute the sum of first N integers using both iterative and direct formula approaches

2. Problem Description

Two methods are used to compute the sum of first N integers ($N = 1024$):

1. Iterative Approach – Sum computed using loop
2. Formula Approach – Using formula $n(n + 1)/2$

Each method is implemented using CUDA kernels.

4. Results

- Iterative Sum = **524800**
- Formula Sum = **524800**

Both methods produced identical results

5. Discussion

Formula-based computation is faster due to $O(1)$ complexity

Iterative approach is slower due to loop operations

GPU overhead dominates for small N

Demonstrates difference between algorithmic complexity

6. Conclusion

CUDA successfully executes multiple kernels performing different tasks. The formula-based approach is more efficient, highlighting the importance of algorithm selection.

Parallel Merge Sort

1. Objective

To implement merge sort using parallel techniques and compare pipelined CPU execution with CUDA-based parallel execution.

2. Problem Description

An array of size 1000 is sorted using:

1. **Pipelined Approach** (CPU-based concept)
2. **CUDA Parallel Sorting**

3. Methodology

1. Generated random array of size 1000
2. Implemented basic sorting using parallel threads
3. Used CUDA kernel to distribute comparisons
4. Compared execution behavior with pipelined approach

4. Results

- Sorted output verified correct
- CUDA execution completed faster than sequential/pipelined approach

5. Discussion

- CUDA exploits parallelism → faster sorting
- Pipelining improves throughput but is limited
- GPU execution reduces computation time
- Memory access patterns affect performance

6. Conclusion

Parallel sorting using CUDA improves performance over CPU-based pipelining, especially for larger datasets.

Vector Addition + Profiling

1. Objective

To implement vector addition in CUDA and analyze performance using profiling tools and bandwidth calculations.

2. Problem Description

Perform vector addition:

$$C[i] = A[i] + B[i]$$

and compute:

- Kernel execution time
- Theoretical memory bandwidth
- Measured memory bandwidth

3. Methodology

1. Declared static arrays
2. Allcated GPU memory
3. Copied data from host to device
4. Launched CUDA kernel
5. Measured execution time using CUDA events
6. Queried device properties using cudaDeviceProp
7. Calculated:
8. Theoretical bandwidth
9. Measured bandwidth

4.Results

Kernel Time: 109.717506 ms

Measured Bandwidth: 0.000112 GB/s

5. Analysis

Theoretical Bandwidth

$$BW = 2 \times memoryClockRate \times memoryBusWidth$$

Measured Bandwidth

$$BW = \frac{RBytes + WBytes}{time}$$

Where:

- Reads = $2N$ floats
- Writes = N floats

5. Performance Discussion

Measured bandwidth < theoretical bandwidth

Difference due to:

Memory latency

Kernel overhead

Non-coalesced memory access

GPU achieves high throughput but not ideal limits

6. Conclusion

CUDA vector addition demonstrates efficient parallel execution. Bandwidth analysis shows real-world limitations compared to theoretical performance. Due to limitations in the CUDA environment, `memoryClockRate` and `memoryBusWidth` were not directly accessible.

Theoretical bandwidth was estimated based on typical GPU specifications